

INF3203 - Distributed Systems
Assignment 2 - Distributed log server

Jens Christian Valen Leynse

jeley4465@uit.no

Tromsø, Norway

UiT

Bjørn Helge Kværnmo

bkv005@uit.no

Tromsø, Norway

UiT

March 24, 2025

1 Introduction

The goal of the assignment is to make a server that manages log entries consistently across the entire system, while also ensuring robustness. The system should be fault-tolerant, meaning that it will continue to function even if an individual node/server experiences failures or communication issues.

The clients should be able to connect to any available server where they will put their log entries. The server is responsible for storing the values and distributing the data consistently and in order across all servers.

At its core, this is a practical implementation of the Paxos consensus algorithm as described in Leslie Lamport's "Paxos Made Simple" paper. Paxos provides the foundation for keeping all servers synchronized through its efficient two-phase protocol, even when some servers temporarily fail or become unreachable. The algorithm automatically handles these real-world challenges while preserving data consistency.

2 Problem Description

The key challenge is creating a distributed log system where:

The core challenge involves building a distributed log system that maintains perfect consistency across all servers while remaining resilient to failures. Such a system must guarantee that every server stores identical log entries in the same order, ensuring clients receive consistent results regardless of which server they connect to. Crucially, the system needs to continue operating normally even when individual servers crash or become unreachable, maintaining availability for both storing and retrieving data as long as a majority of servers remain operational.

From a technical perspective, this requires ensuring safety properties where only one valid value can ever be accepted for each log position, while also maintaining liveness so the system continues making progress when most servers are available. The solution must properly handle failures, typically tolerating up to f server failures in a system with $2f+1$ servers. In practical terms, this means if you store a log entry through any server, you should later be able to retrieve that exact same entry through any other functioning server - even if some servers are temporarily unavailable. The system must automatically manage these failure scenarios without any data loss or inconsistent results, providing seamless continuity of service.

3 Method

The implementation follows Lamport's two-phase protocol:

3.1 Phase 1: Prepare-Promise

- Proposer selects unique proposal number n
- Sends $\text{PREPARE}(n)$ to majority of acceptors
- Acceptor responds with:
 - $\text{PROMISE}(n, (n_a, v_a))$ where (n_a, v_a) is the highest-numbered proposal accepted
 - Or rejects if it promised to ignore proposals $\leq n$

3.2 Phase 2: Accept-Accepted

- If proposer receives PROMISE from majority:
 - Selects value v from highest-numbered response or new value
 - Sends $\text{ACCEPT}(n, v)$ to acceptors
- Acceptor accepts if no promise made to higher n
- Upon acceptance by majority, value is chosen

4 Experiments and Results

Our testing revealed fundamental consistency challenges across all scenarios (0-4) from the client. While Paxos theoretically guarantees safety, our practical implementation showed limitations:

4.1 Test Scenarios

We implemented five progressively challenging test cases:

- **Scenario 0:** Baseline test with 20 log entries and infrequent, non-overlapping crashes (1-2 nodes down for 5-15s)
- **Scenario 1:** Moderate workload (20 entries) with single-node crashes every 5-7 seconds
- **Scenario 2:** High-intensity test (100 entries) with frequent crashes (every 1-2s) and short downtimes (3-4s)
- **Scenario 3:** Extended test (100 entries) with multiple concurrent node failures (2 nodes down simultaneously)
- **Scenario 4:** Worst-case scenario testing maximum fault tolerance (majority of nodes failing simultaneously)

4.2 Key Findings

- **All scenarios exhibited missing entries** - Even Scenario 0 (no forced crashes) showed 5-15% missing entries due to Python's GIL and network timing issues
- **Crash-heavy scenarios (3-4)** - Up to 35% of entries were missing on recovered nodes, with inconsistencies growing with crash frequency
- **Pattern observation** - Nodes consistently missed entries proposed during their crashed periods, confirming the lack of a recovery protocol

5 Discussion

The experimental results expose fundamental limitations in maintaining consistency during node failures, despite correctly implementing Paxos' core safety properties to prevent conflicting values. The system's inability to achieve complete log consistency stems from two critical architectural gaps.

First, recovered nodes lack any mechanism to synchronize missed entries after crashes, particularly when failures occur during value acceptance phases. This explains the observed discrepancies where nodes processing identical operations produced logs of varying lengths (49-64 entries in Scenario 4's 79-operation test case). Second, the transient in-memory storage of accepted values in the Learner class proves inadequate, as crash events permanently erase this critical state information.

Our decision to implement basic Paxos rather than more complex alternatives was driven by its mathematically rigorous safety proofs, relatively simple two-phase protocol compared to Multi-Paxos variants, and sufficient core consensus mechanism for log ordering requirements. However, production-grade deployments would require extending this foundation with robust recovery mechanisms. Potential solutions include periodic state transfers where recovered nodes reconcile differences with peers, persistent acceptance receipts for tracking committed values, or hybrid approaches combining write-ahead logging with state synchronization. Each option presents distinct tradeoffs between implementation complexity, recovery performance, and storage overhead that would need evaluation based on specific operational requirements. Ultimately, these results demonstrate that while Paxos provides an excellent theoretical foundation, practical systems must incorporate additional recovery protocols to handle the inevitable message loss during node failures.

6 Conclusion

All test scenarios ultimately failed to maintain perfect consistency, revealing a crucial gap between theoretical design and practical implementation. While our system correctly executed Paxos' two-phase protocol for active nodes, it lacked three essential production-grade features: persistent storage of accepted values, state synchronization for recovered nodes, and proper handling of transient network issues. The experimental results clearly show that while Paxos provides an excellent foundation for distributed consensus, operational deployments demand additional layers of fault tolerance beyond the core algorithm.

Reference:

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/paxos-simple-Copy.pdf>